Sprint Ceremony Document: Sprint 1

Project Name: ALT-F4
Sprint Window: October 11 – October 17, 2026
Overall Completion: 23/168 * 100 = **13.69%**

Sprint Planning & Progress Update

Completed Tasks:

- **Web Hosting & Deployment Setup:** Provisioned and configured web hosting infrastructure for initial application deployment.
- **Backend Core Structure:** Implemented base API routing and database connection scripts according to specified schemas.
- **Frontend UI Foundation:** Established core page layouts and initial component structures based on design mockups.

Planned Tasks for Next Sprint (Sprint 2):

Sprint 2

Task ID	Task Title	Description	Assignee(s)
CQ-S2-2.1	Implement authentication and profile APIs	Implement registration, login, logout and profile behavior based on the supplied API specification; validate inputs and handle authentication errors.	John Albert Masangkay; Matthew Ryan Sabino
CQ-S2-2.2	Enforce role-based access control	Enforce separate affiliation and permission role concepts; protect frontend routes and backend endpoints for patient, clinic staff and super admin.	John Albert Masangkay; Matthew Ryan Sabino
CQ-S2-2.3	Build registration and login screens	Implement responsive forms from the supplied mockups and connect them to real authentication APIs.	Jereign Lim; Matthew Ryan Sabino supports integration
CQ-S2-2.4	Build patient and clinic-staff dashboard shells	Build role-appropriate responsive dashboard shells and navigation with safe loading, empty and error states.	Jereign Lim
CQ-S2-2.5	Integrate authentication with protected dashboards	Integrate API responses with application auth state, protected routes, session handling and logout behavior.	Matthew Ryan Sabino; John Albert Masangkay; Jereign Lim
CQ-S2-2.6	Test authentication, roles and dashboard integration	Execute authentication and cross-role test scenarios, fix critical issues and capture the sprint demo evidence.	John Alec Resos; John Raphael Reganit; developers resolve defects

Sprint Retrospective

What Went Well:

- Infrastructure setup and initial web hosting environment were configured on schedule.
- Clear separation of developer responsibilities between API implementation and frontend components.
- Tech stack was researched and improved

What Could Be Improved:

- Problems found were searching for an API that is suitable and with no to minimal cost.

What Needs to Be Done:

- Proceed with the implementation of user roles and authentication.
- Start with the creation of dashboard components

Individual Reports

Sprint: CQ-S1-1.1

Member: John Albert D. Masangkay

Role: Tech Lead

Task: Review and prioritize requirements, translate the proposal, user stories, and use cases into actor workflows, business rules, acceptance checks, unresolved decisions and explicit scope exclusions.

Done:

- Reviewed the proposal, user stories, and use case diagram, then extracted the user visible actions for the patient, clinic staff and admin.
- Defined the appointment and queue states, and listed the action that create an audit entry.
- Set the key business rules, no booking of full or unpublished slots, no cancellation within 2 hours, same day queue only, no duplicate queue entries and no editing of audit entries.
- Wrote Given / When / Then acceptance checks for accounts, scheduling, booking, queue, consultation and audit.

Sprint: CQ-S1-1.2

Member: All Members

Role: All Roles

Task: Define and document the project's technology stack, repository structure, development tools, security practices, testing approach, and deployment architecture to establish a consistent foundation for collaborative development and future feature integration.

Done:

- Finalized the technology stack, using Angular with TypeScript and SCSS for the frontend, PrimeNG for UI components, and PrimeIcons for interface icons.
- Defined the backend architecture using Node.js (LTS) and NestJS with REST APIs, Swagger/OpenAPI documentation, and Socket.IO for real-time queue updates and patient-call notifications.
- Selected PostgreSQL through Supabase for data storage and Prisma ORM for database queries, schema management, and migrations.
- Established the authentication and security approach using Passport/JWT for authenticated API access and Argon2 for password hashing.
- Identified Postman and Playwright as the primary tools for API testing and end-to-end workflow validation, respectively.
- Defined a monorepo structure to organize the frontend, backend, documentation, and configuration files in a single repository.
- Established Git and GitHub as the version control and repository hosting platforms, with feature branches to support collaborative development, code reviews, bug fixes, and integration.
- Specified environment configuration practices using `.env` and `.env.example` files to manage application settings while preventing sensitive credentials from being committed to version control.
- Documented the deployment architecture, using Vercel for frontend hosting and Render for backend hosting, to support the deployment and integration of the application's components.

Sprint: CQ-S1-1.3

Member: Matthew Ryan R. Sabino
Role: Project Manager
Task: Deploy the application using Vercel and Render, document the hosted URLs, and make the `/display` route available for deployment checks, accessibility testing, and integration validation across the hosted environments.

Done:

- Published the application to Vercel and documented the deployment URL: https://project-altf4.vercel.app/display.
- Established the Render deployment URL: https://project-altf4.onrender.com/display.
- Documented both hosted endpoints to provide clear access points for reviewing the deployed application and checking route behavior.
- Established the hosted environments as references for subsequent deployment validation, frontend-backend integration checks, and end-to-end workflow testing.
- Recorded the deployment URLs for inclusion in project documentation and use during development reviews and testing.

Sprint: CQ-S1-1.4

Member: John Albert D. Masangkay, Matthew Ryan R. Sabino

Role: Technical Lead

Task: Review database schema, review the ERD and relationships for app_user, clinic, daily_queue, appointment, queue_entry, notification and audit_log; document integrity rules before feature work

Done:

- Defined the tables and their fields: app_user, clinic_slot, daily_queue, appointment, queue_entry, notification and audit_log.
- Mapped the primary keys, foreign keys and relationships between the tables (for example, appointment to app_user and clinic_slot, queue_entry to daily_queue and appointment).
- Defined the status and role types used by the tables, such as appointment status, queue status, notification type and access role
- .

Sprint: CQ-S1-1.5

Member: Jereign B. Lim

Role: UI/UX Designer / Developer

Task: Map supplied mockups to the application route/component inventory, including patient, staff and super-admin screens.

Done:

- Planned the UI before coding starts in Angular.
- Made a screen and route inventory of 15 screens (3 public, 5 patient, 4 staff, 3 admin)
- Grouped routes by portal, so access can be controlled by role.
- Planned 4 shared layouts (Auth, Patient, Staff, Admin) and the reusable components (buttons, form fields, cards, tables, badges, alerts), so the team does not duplicate code.
- Defined responsive breakpoints (mobile, tablet, desktop)
- Defined loading, empty, validation, error and success states for every screen, so no page is blank or broken.
- Combined everything into one mapping table that shows which layout and components each screen uses.

Next sprint:

- Registration and login screens, with Matthew on API integration.
- Patient and clinic-staff dashboard shells with responsive, loading, empty and error states.

Sprint: CQ-S1-1.6

Member: John Alec L. Resos, John Raphael Reganit

Role: Tester | Developer

Task: List which actions patient, clinic staff and admin should be permitted or denied, and identify fictional test data needed.

Done:

- Created 2 tables that specify the actions that are permitted and not permitted for patients and staff.
- Identified the fictional data needed.
- Assisted in choosing hosting platforms and deploying the initial frontend & backend template.